Computational Foundations and Matrix Methods

These notes combine two introductory classes into one coherent unit for a doctoral course in biostatistical computing. The central theme is that a statistical method is not complete until its mathematical objective has been translated into a numerically stable, reproducible, and diagnostically transparent implementation. Examples use R, but the same principles apply in Python, Julia, MATLAB, and C++.

1 Course scope and teaching sequence

Learning objectives.

- Translate statistical estimators into computational objectives and linear-algebra operations.
- Distinguish mathematically equivalent formulas that have very different numerical stability, memory use, or runtime.
- Use matrix factorizations to solve regression, covariance, simulation, and dimension-reduction problems without unnecessary inversion.
- Implement and diagnose PCA, SVD, CCA, matrix factorization, k-means, and multidimensional scaling.
- Connect computational choices to applications in genomics, neuroimaging, multi-omics, and longitudinal biomedical studies.

1.1 Prerequisites and computing environment

Students should have working knowledge of calculus, probability, statistical inference, and introductory linear algebra. Prior experience with at least one programming language is expected. R and RStudio are sufficient for the examples. Useful extensions include Rcpp and RcppArmadillo for compiled code, Git and GitHub for version control, and a high-performance computing system for large simulations or high-dimensional biomedical data.

The following books are useful throughout the course:

- Monahan, *Numerical Methods of Statistics*;
- Lange, *Numerical Analysis for Statisticians*;
- Gentle, *Computational Statistics*;
- Golub and Van Loan, *Matrix Computations*; and
- Higham, *Accuracy and Stability of Numerical Algorithms*.

2 From a scientific question to a computational method

A biostatistical analysis usually follows the chain

scientific question and data-generating mechanism
$\Longrightarrow$ probability model or estimating equation
$\Longrightarrow$ computational objective and algorithm
$\Longrightarrow$ diagnostics, uncertainty assessment, and reproducible output.

Each arrow involves scientific and computational judgment. A software call is therefore the end of a reasoning process, not a substitute for it.

Many estimators can be written as

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}\in\Theta} Q_n(\boldsymbol{\theta}),$$

where Q_n may be a negative log-likelihood, a loss plus a penalty, or a distance from an estimating equation to zero. The parameter space Θ also matters: it may impose nonnegativity, sparsity, positive definiteness, or other scientifically motivated constraints.

Example 1 (Least squares and constrained variants)**.** *For a linear model, ordinary least squares minimizes*

$$Q_n(\boldsymbol{\beta}) = \|\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2, \qquad \boldsymbol{\beta} \in \mathbb{R}^p.$$

Nonnegative least squares changes only the feasible set to $\mathbb{R}^p_+$. Ridge regression changes the objective to

$$\|\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2.$$

The scientific models are closely related, but the algorithms and diagnostics differ. Here $\|\mathbf{a}\|_2^2 = \sum_i a_i^2$.

Understanding the algorithm helps a biostatistician:

- recognize nonconvergence, rank deficiency, separation, or an ill conditioned information matrix;
- choose an algorithm whose cost is appropriate for n, p, and the structure of the data;
- distinguish optimization error from statistical uncertainty;
- design new estimators instead of being limited to existing software;
- verify theoretical results by simulation; and
- distribute code that others can inspect and reproduce.

3 Vectorization, memory, and computational cost

3.1 What vectorization does

R and Python execute many array operations in optimized compiled libraries. Replacing an interpreted loop with a vectorized operation can reduce interpreter overhead and may use optimized BLAS, cache-aware kernels, or parallel hardware.

```r
set.seed(1)
a <- rnorm(100000)
b <- rnorm(100000)

# Vectorized
c_vec <- a + b

# Explicit loop
c_loop <- numeric(length(a))
for (i in seq_along(a)) {
  c_loop[i] <- a[i] + b[i]
}

stopifnot(all.equal(c_vec, c_loop))
```

Vectorization is not a rule that loops are always wrong. A vectorized expression may allocate a very large temporary object, and some iterative algorithms are naturally expressed as loops. The real goal is to select an implementation with reasonable runtime, memory use, and numerical behavior.

3.2 Memory layout and aliasing

R matrices use column-major storage. NumPy arrays are row-major by default, but can also use Fortran-style column-major storage. This affects the order in which elements are stored and can influence cache efficiency. An R vector does not have a row or column orientation until it is used in a matrix operation or given matrix dimensions.

In Python, assigning one NumPy array to another name usually creates a second reference to the same data:

```python
import numpy as np

x = np.array([2.0])
y = x
x[0] = 3.0
print(y)          # y also shows 3.0

z = x.copy()
x[0] = 4.0
print(z)          # z remains 3.0
```

This behavior differs from assigning immutable Python scalars. Similar copy-versus-reference issues arise in R, particularly when objects are modified inside functions or through external-memory interfaces.

3.3 Count both floating-point operations and bytes

Multiplying an $m \times p$ matrix by a $p \times n$ matrix produces mn entries. Each entry uses p multiplications and $p - 1$ additions, so the classical algorithm uses

$$mn(2p - 1)$$

floating-point operations. This count explains why exploiting structure is important.

Example 2 (Do not store a diagonal matrix when a vector is enough)**.** *If* $\mathbf{D} = \operatorname{diag}(d_1, \ldots, d_p)$*, then* $\mathbf{D}^2 = \operatorname{diag}(d_1^2, \ldots, d_p^2)$*. Constructing a dense* $p \times p$ *matrix and multiplying it by itself wastes both memory and computation.*

```
set.seed(1)
d <- rnorm(2000)

system.time(D2_fast <- d^2)

# Dense version shown only for comparison
D <- diag(d)
system.time(D2_dense <- D %*% D)

stopifnot(all.equal(diag(D2_dense), D2_fast))
```

Likewise, to compute $\mathbf{A} = \mathbf{B} + \operatorname{diag}(\boldsymbol{d})$ for a dense square matrix $\mathbf{B}$, update only the diagonal:

```
A <- B
diag(A) <- diag(A) + d
```

3.4 Useful structured operations in R

Task	Preferred R expression	Comment
$\mathbf{X}^\top\mathbf{X}$	`crossprod(X)`	Avoids writing an explicit transpose.
$\mathbf{X}\mathbf{X}^\top$	`tcrossprod(X)`	Often useful for $p \gg n$.
Row or column sums	`rowSums(X)`, `colSums(X)`	Faster and clearer than `apply` for this task.
Solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$	`solve(A, b)`	Do not compute `solve(A) %*% b`.
Triangular solve	`forwardsolve`, `backsolve`	Exploit a known factorization.
Sparse operations	`Matrix` package	Store only nonzero entries when sparsity is substantial.

Computing note

Benchmark realistic inputs after a short warm-up, and verify that competing implementations return the same numerical result. Runtime alone is not enough: also check peak memory, convergence, and sensitivity to ill conditioning.

4 Linear-algebra essentials

4.1 Notation and dimensions

Scalars are written as a, vectors as $\boldsymbol{a}$, and matrices as $\mathbf{A}$. If $\mathbf{A} \in \mathbb{R}^{m\times p}$ and $\mathbf{B} \in \mathbb{R}^{p\times n}$, then $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m\times n}$, with

$$c_{ij} = \sum_{k=1}^{p} a_{ik} b_{kj}.$$

Checking dimensions before coding catches many errors.

Several products occur frequently:

- the inner product, $\boldsymbol{a}^\top \boldsymbol{b} = \sum_i a_i b_i$;
- the outer product, $\boldsymbol{a}\boldsymbol{b}^\top$;
- the Hadamard product, $(\mathbf{A} \odot \mathbf{B})_{ij} = a_{ij} b_{ij}$, for matrices of the same size; and
- the Kronecker product, $\mathbf{A} \otimes \mathbf{B}$, a block matrix that appears in longitudinal, spatial, and multivariate covariance models.

If $\mathbf{A} = [\boldsymbol{a}_1, \dots, \boldsymbol{a}_p]$ and the rows of $\mathbf{B}$ are $\boldsymbol{b}_1^\top, \dots, \boldsymbol{b}_p^\top$, then

$$\mathbf{AB} = \sum_{k=1}^{p} \boldsymbol{a}_k \boldsymbol{b}_k^\top.$$

This outer-product view is the basis of low-rank approximation and many parallel matrix algorithms.

4.2 Trace identities and norms

For conformable matrices,

$$\begin{aligned} \operatorname{tr}(\mathbf{A} + \mathbf{B}) &= \operatorname{tr}(\mathbf{A}) + \operatorname{tr}(\mathbf{B}), \\ \operatorname{tr}(\mathbf{AB}) &= \operatorname{tr}(\mathbf{BA}), \\ \operatorname{tr}(\mathbf{ABC}) &= \operatorname{tr}(\mathbf{BCA}) = \operatorname{tr}(\mathbf{CAB}), \\ \operatorname{tr}(\mathbf{A}^\top \mathbf{B}) &= \sum_{i,j} a_{ij} b_{ij}. \end{aligned}$$

The Frobenius norm therefore satisfies

$$\|\mathbf{A}\|_F^2 = \operatorname{tr}(\mathbf{A}^\top \mathbf{A}) = \sum_{i,j} a_{ij}^2.$$

The spectral norm is $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$, where $\sigma_{\max}$ is the largest singular value.

4.3 Positive definiteness

A real symmetric matrix $\mathbf{A}$ is positive definite if

$$\boldsymbol{x}^\top \mathbf{A} \boldsymbol{x} > 0 \quad \text{for every } \boldsymbol{x} \neq \mathbf{0},$$

and positive semidefinite if the inequality is non-strict. Positive definite matrices have strictly positive eigenvalues; positive semidefinite matrices have nonnegative eigenvalues.

Covariance matrices are always positive semidefinite, but need not be positive definite. For example, exact linear dependence among variables produces a singular covariance matrix. This distinction determines whether a Cholesky factorization or an ordinary inverse exists.

5 Linear systems and matrix factorizations

The recurring problem in statistical computing is not to calculate $\mathbf{A}^{-1}$, but to solve

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b}.$$

Explicit inversion is usually slower, uses more memory, and can accumulate more rounding error than a factorization followed by triangular solves.

5.1 Triangular systems

If $\mathbf{L}$ is lower triangular, forward substitution solves $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$. If $\mathbf{R}$ is upper triangular, back substitution solves $\mathbf{R}\boldsymbol{x} = \boldsymbol{y}$. Most dense factorizations reduce a general system to one or more triangular systems.

5.2 A factorization guide

Factorization	Matrix class	Typical use
$\mathbf{A} = \mathbf{LU}$ with pivoting	General square matrix	Solve nonsymmetric systems; evaluate determinants.
$\mathbf{X} = \mathbf{QR}$	Rectangular or square matrix	Least squares, orthogonalization, rank detection.
$\mathbf{A} = \mathbf{LL}^{\mathsf{T}}$	Symmetric positive definite matrix	Covariance systems, Gaussian simulation, log determinants.
$\mathbf{X} = \mathbf{UDV}^{\mathsf{T}}$	Any rectangular matrix	Rank, pseudoinverse, low-rank approximation, PCA.

In production software, LU is normally used with row pivoting. QR is generally computed by Householder transformations rather than classical Gram–Schmidt, because Householder QR has better numerical behavior. Cholesky is especially efficient, but its positive-definiteness requirement must be checked.

5.3 Worked Cholesky recursion

Suppose $\mathbf{A} = \mathbf{LL}^{\mathsf{T}}$, where $\mathbf{A}$ is symmetric positive definite and $\mathbf{L}$ is lower triangular. The first entries satisfy

$$\ell_{11} = \sqrt{a_{11}}, \qquad \ell_{21} = \frac{a_{21}}{\ell_{11}}, \qquad \ell_{22} = \sqrt{a_{22} - \ell_{21}^2}.$$

In general,

$$\ell_{jj} = \left(a_{jj} - \sum_{k=1}^{j-1} \ell_{jk}^2 \right)^{1/2},$$

$$\ell_{ij} = \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \ell_{ik}\ell_{jk} \right), \qquad i > j.$$

A Cholesky routine fails when a computed diagonal pivot is nonpositive. This may indicate that the matrix is not positive definite, that it is singular, or that rounding error has exposed severe ill conditioning. Adding a small diagonal constant can be scientifically justified in some regularized models, but should not be used to conceal a data or coding error.

Once $\mathbf{A} = \mathbf{LL}^{\mathsf{T}}$,

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b} \quad \Longleftrightarrow \quad \mathbf{L}\boldsymbol{y} = \boldsymbol{b}, \qquad \mathbf{L}^{\mathsf{T}}\boldsymbol{x} = \boldsymbol{y}.$$

```
# R returns an upper-triangular R with A = t(R) %*% R
Rfac <- chol(A)
y <- forwardsolve(t(Rfac), b)
x <- backsolve(Rfac, y)

# Equivalent high-level call
```

```r
x_check <- solve(A, b)
stopifnot(all.equal(x, x_check))
```

5.4 Gaussian simulation

If $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$ and $\boldsymbol{z} \sim N_p(\mathbf{0}, \mathbf{I})$, then

$$\boldsymbol{x} = \boldsymbol{\mu} + \mathbf{L}\boldsymbol{z} \sim N_p(\boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

Because R's `chol` returns an upper-triangular factor $\mathbf{R}$ satisfying $\boldsymbol{\Sigma} = \mathbf{R}^\top\mathbf{R}$, the R implementation uses $\mathbf{R}^\top\boldsymbol{z}$.

```r
Rfac <- chol(Sigma)
z <- rnorm(nrow(Sigma))
x <- drop(mu + t(Rfac) %*% z)
```

5.5 Conditioning

The condition number

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \left\|\mathbf{A}^{-1}\right\|_2 = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$$

measures the sensitivity of a nonsingular linear system to perturbations. Large condition numbers mean that small changes in the data or rounding errors can produce large changes in the solution. A stable algorithm cannot recover information that the mathematical problem itself does not contain.

6 Regression as numerical linear algebra

Let

$$\boldsymbol{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}, \qquad \mathbf{X} \in \mathbb{R}^{n \times p}.$$

The least-squares objective is

$$Q(\boldsymbol{\beta}) = (\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^\top(\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}).$$

Its gradient is

$$\nabla_{\boldsymbol{\beta}} Q(\boldsymbol{\beta}) = 2\mathbf{X}^\top(\mathbf{X}\boldsymbol{\beta} - \boldsymbol{y}),$$

so the normal equations are

$$\mathbf{X}^\top\mathbf{X}\,\widehat{\boldsymbol{\beta}} = \mathbf{X}^\top\boldsymbol{y}.$$

These equations characterize the solution, but they do not prescribe the best way to compute it.

6.1 QR solution

For a full-column-rank design, write the thin QR factorization $\mathbf{X} = \mathbf{Q}\mathbf{R}$, where $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}_p$ and $\mathbf{R}$ is $p \times p$ upper triangular. Then

$$\widehat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left\|\mathbf{Q}^\top\boldsymbol{y} - \mathbf{R}\boldsymbol{\beta}\right\|_2^2, \qquad \mathbf{R}\widehat{\boldsymbol{\beta}} = \mathbf{Q}^\top\boldsymbol{y}.$$

Thus QR solves one triangular system directly. Forming $\mathbf{X}^\top\mathbf{X}$ is less desirable because

$$\kappa_2(\mathbf{X}^\top\mathbf{X}) = \kappa_2(\mathbf{X})^2,$$

which squares the conditioning problem.

```r
X <- model.matrix(~ age + sex + treatment, data = dat)
y <- dat$outcome

qrx <- qr(X)
beta_qr <- qr.coef(qrx, y)

# Pedagogical comparison only; avoid as the default algorithm
beta_normal <- solve(crossprod(X), crossprod(X, y))
```

Pivoted QR or SVD should be used when the design is rank deficient or nearly rank deficient. Statistical software may omit aliased coefficients or return generalized solutions; the analyst must identify the underlying design issue.

6.2 Generalized least squares

If $\operatorname{Cov}(\boldsymbol{\epsilon}) = \boldsymbol{\Sigma}$, generalized least squares minimizes

$$(\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{y} - \mathbf{X}\boldsymbol{\beta}).$$

One should not explicitly invert $\boldsymbol{\Sigma}$. If $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^{\mathsf{T}}$, solve

$$\widetilde{\boldsymbol{y}} = \mathbf{L}^{-1}\boldsymbol{y}, \qquad \widetilde{\mathbf{X}} = \mathbf{L}^{-1}\mathbf{X},$$

and perform ordinary least squares on $(\widetilde{\boldsymbol{y}}, \widetilde{\mathbf{X}})$. This whitening step appears in longitudinal models, spatial models, and correlated neuroimaging outcomes.

7 Matrix calculus used in statistical computing

The differential form of matrix calculus is compact and helps verify gradients. For an invertible matrix $\mathbf{A}$,

$$d \log\det(\mathbf{A}) = \operatorname{tr}(\mathbf{A}^{-1}d\mathbf{A}), \qquad d(\mathbf{A}^{-1}) = -\mathbf{A}^{-1}(d\mathbf{A})\mathbf{A}^{-1}.$$

For constant matrices and vectors of compatible dimensions,

$$\begin{aligned}
\nabla_{\boldsymbol{x}}(\boldsymbol{a}^{\mathsf{T}}\boldsymbol{x}) &= \boldsymbol{a}, \\
\nabla_{\boldsymbol{x}}(\boldsymbol{x}^{\mathsf{T}}\mathbf{A}\boldsymbol{x}) &= (\mathbf{A} + \mathbf{A}^{\mathsf{T}})\boldsymbol{x}, \\
\nabla_{\mathbf{X}} \operatorname{tr}(\mathbf{A}^{\mathsf{T}}\mathbf{X}) &= \mathbf{A}.
\end{aligned}$$

When $\mathbf{A}$ is symmetric, the second identity becomes $2\mathbf{A}\boldsymbol{x}$.

These formulas underlie gradients of Gaussian log-likelihoods. For example, apart from constants,

$$\ell(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{1}{2}\log\det(\boldsymbol{\Sigma}) - \frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}).$$

In computation, both the log determinant and the quadratic form can be evaluated from a Cholesky factor without constructing $\boldsymbol{\Sigma}^{-1}$.

Computing note

Before trusting an analytic gradient, compare it with a finite-difference approximation at several non-pathological parameter values. Finite differences are a diagnostic, not a replacement for a stable analytic or automatic derivative.

8 Spectral decomposition

For a real symmetric matrix $\mathbf{A}$, the spectral theorem gives

$$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{U}^\mathsf{T} = \sum_{j=1}^{p} \lambda_j \boldsymbol{u}_j \boldsymbol{u}_j^\mathsf{T},$$

where $\mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{I}$ and $\mathbf{D}$ is diagonal. This orthogonal form is not available for every square matrix; a general square matrix may have a nonorthogonal or even incomplete eigenvector system.

The eigenvalue equation is

$$\mathbf{A}\boldsymbol{u}_j = \lambda_j \boldsymbol{u}_j.$$

For any square matrix, counting eigenvalues with algebraic multiplicity,

$$\operatorname{tr}(\mathbf{A}) = \sum_j \lambda_j, \qquad \det(\mathbf{A}) = \prod_j \lambda_j.$$

For symmetric $\mathbf{A}$, functions defined on its eigenvalues can be evaluated spectrally:

$$f(\mathbf{A}) = \mathbf{U} f(\mathbf{D}) \mathbf{U}^\mathsf{T}.$$

This includes powers, exponentials, and, when the eigenvalues are positive, matrix logarithms and fractional powers. The function must be well defined on the spectrum; analyticity is sufficient but not always necessary.

8.1 QR iteration

The basic unshifted QR iteration repeatedly computes

$$\mathbf{A}_{k-1} = \mathbf{Q}_k \mathbf{R}_k, \qquad \mathbf{A}_k = \mathbf{R}_k \mathbf{Q}_k = \mathbf{Q}_k^\mathsf{T} \mathbf{A}_{k-1} \mathbf{Q}_k.$$

Thus every iterate is orthogonally similar to the original matrix and has the same eigenvalues. For a symmetric matrix, the off-diagonal entries tend toward zero under suitable conditions.

```
set.seed(1)
Z <- matrix(rnorm(100), 10, 10)
A0 <- (Z + t(Z)) / 2

Ak <- A0
U <- diag(nrow(Ak))
tol <- 1e-10

for (iter in 1:5000) {
  qra <- qr(Ak)
  Q <- qr.Q(qra)
  R <- qr.R(qra)
  Ak <- R %*% Q
  U <- U %*% Q

  offdiag <- sqrt(sum(Ak[lower.tri(Ak)]^2))
  if (offdiag < tol) break
}

sort(diag(Ak))
sort(eigen(A0, symmetric = TRUE)$values)
```

Production eigensolvers first reduce the matrix to a simpler form and use shifts, deflation, and carefully designed stopping rules. The simple code is for understanding the algorithm, not replacing a library routine.

9 Singular value decomposition

Every $\mathbf{X} \in \mathbb{R}^{n \times p}$ has a singular value decomposition

$$\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^{\mathsf{T}}.$$

In the economy form, if $r = \operatorname{rank}(\mathbf{X})$, $\mathbf{U} \in \mathbb{R}^{n \times r}$, $\mathbf{D} \in \mathbb{R}^{r \times r}$, and $\mathbf{V} \in \mathbb{R}^{p \times r}$. The diagonal entries

$$\sigma_1 \geq \cdots \geq \sigma_r > 0$$

are the singular values. The columns of $\mathbf{U}$ and $\mathbf{V}$ are the left and right singular vectors.

Important consequences include:

- $\|\mathbf{X}\|_2 = \sigma_1$ and $\|\mathbf{X}\|_F^2 = \sum_j \sigma_j^2$;
- the number of nonzero singular values equals the rank;
- the Moore–Penrose pseudoinverse is $\mathbf{X}^+ = \mathbf{V}\mathbf{D}^+\mathbf{U}^{\mathsf{T}}$; and
- the best rank-K approximation in spectral or Frobenius norm is

$$\mathbf{X}_K = \sum_{j=1}^{K} \sigma_j \boldsymbol{u}_j \boldsymbol{v}_j^{\mathsf{T}}.$$

Numerical rank depends on a tolerance. Reciprocating extremely small singular values can amplify noise, so pseudoinverse routines treat singular values below a scale-dependent threshold as zero. The threshold is part of the statistical regularization and should not be viewed as a purely technical detail.

9.1 Why SVD is central for high-dimensional biostatistics

When $p \gg n$, forming a $p \times p$ covariance matrix can be wasteful. The SVD of the centered $n \times p$ data matrix directly provides principal directions, scores, rank information, and low-rank reconstructions. This is especially useful for gene-expression, methylation, metabolomics, and imaging features.

10 Principal component analysis

Let $\mathbf{X}_c$ be an $n \times p$ matrix whose columns have been centered. The first principal direction solves

$$\widehat{\boldsymbol{u}}_1 = \underset{\|\boldsymbol{u}\|_2 = 1}{\arg\max} \operatorname{Var}(\mathbf{X}_c \boldsymbol{u}),$$

and later directions satisfy the same criterion subject to orthogonality to earlier directions. If

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}_c^{\mathsf{T}} \mathbf{X}_c = \mathbf{U} \operatorname{diag}(\lambda_1, \ldots, \lambda_p) \mathbf{U}^{\mathsf{T}},$$

with $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$, the score matrix for the first K components is

$$\mathbf{Z}_K = \mathbf{X}_c \mathbf{U}_K.$$

The proportion of sample variance explained by the first K components is

$$\frac{\sum_{j=1}^{K} \lambda_j}{\sum_{j=1}^{p} \lambda_j}.$$

Equivalently, PCA provides the rank-K linear reconstruction minimizing $\left\| \mathbf{X}_c - \widehat{\mathbf{X}}_{c,K} \right\|_F^2$.

10.1 PCA in R: package and from scratch

```
X <- as.matrix(iris[, 1:4])
Xz <- scale(X, center = TRUE, scale = TRUE)

# Package computation
pca <- prcomp(Xz, center = FALSE, scale. = FALSE)
scores <- pca$x[, 1:2]
pve <- pca$sdev^2 / sum(pca$sdev^2)

# Spectral computation from scratch
S <- crossprod(Xz) / (nrow(Xz) - 1)
es <- eigen(S, symmetric = TRUE)
scores_eigen <- Xz %*% es$vectors[, 1:2]

# Eigenvectors have arbitrary signs, so compare up to sign
abs(cor(scores[, 1], scores_eigen[, 1]))
abs(cor(scores[, 2], scores_eigen[, 2]))

plot(scores[, 1], scores[, 2],
     col = as.numeric(iris$Species), pch = 19,
     xlab = sprintf("PC1: %.1f%%", 100 * pve[1]),
     ylab = sprintf("PC2: %.1f%%", 100 * pve[2]))
legend("topright", legend = levels(iris$Species),
       col = seq_along(levels(iris$Species)), pch = 19)
```

The species labels are used only after PCA to color the plot. PCA itself is unsupervised and does not use those labels.

10.2 Centering, scaling, and study design

Centering is part of ordinary PCA. Scaling to unit variance is a scientific choice:

- scale when variables are measured in incomparable units or when their magnitudes should not determine their influence;
- do not scale automatically when absolute variation is scientifically meaningful; and
- in prediction studies, estimate centering, scaling, and PCA directions using the training data only, then apply them to validation data.

In biomedical data, leading components may reflect batch, site, scanner, or cell-composition effects rather than the biological signal of interest. PCA is a descriptive transformation, not a guarantee of scientific relevance.

10.3 When linear PCA is inadequate

PCA captures a linear subspace. A curved low-dimensional structure, such as a Swiss roll, may require nonlinear methods. Kernel PCA begins with a positive semidefinite Gram matrix among observations,

$$K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j).$$

For a Gaussian radial-basis kernel,

$$k(\boldsymbol{x}_i, \boldsymbol{x}_j) = \exp\left\{ -\frac{\|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2^2}{2h^2} \right\}.$$

The Gram matrix must be centered in feature space:

$$\mathbf{K}_c = \mathbf{H}\mathbf{K}\mathbf{H}, \qquad \mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{1}\mathbf{1}^{\mathsf{T}}.$$

Eigenvectors of $\mathbf{K}_c$ yield nonlinear components. Kernels need not all be distance based. Bandwidth selection, out-of-sample extension, and sensitivity to preprocessing must be addressed explicitly.

11 Canonical correlation analysis

Suppose two centered data blocks are measured on the same n subjects:

$$\mathbf{X} \in \mathbb{R}^{n \times p}, \qquad \mathbf{Y} \in \mathbb{R}^{n \times q}.$$

CCA finds linear combinations with maximal cross-block correlation:

$$(\widehat{\boldsymbol{a}}, \widehat{\boldsymbol{b}}) = \arg\max_{\boldsymbol{a}, \boldsymbol{b}} \boldsymbol{a}^{\mathsf{T}} \mathbf{S}_{XY} \boldsymbol{b}$$

subject to

$$\boldsymbol{a}^{\mathsf{T}} \mathbf{S}_{XX} \boldsymbol{a} = 1, \qquad \boldsymbol{b}^{\mathsf{T}} \mathbf{S}_{YY} \boldsymbol{b} = 1.$$

If the within-block covariance matrices are nonsingular, form the whitened cross-covariance

$$\mathbf{C} = \mathbf{S}_{XX}^{-1/2} \mathbf{S}_{XY} \mathbf{S}_{YY}^{-1/2} = \mathbf{U}\mathbf{D}\mathbf{V}^{\mathsf{T}}.$$

The singular values in $\mathbf{D}$ are the canonical correlations, and the canonical coefficient vectors are based on $\mathbf{S}_{XX}^{-1/2}\mathbf{U}$ and $\mathbf{S}_{YY}^{-1/2}\mathbf{V}$.

CCA is useful for paired data blocks such as gene expression and metabolomics, imaging and cognition, or two assay platforms. Three cautions are essential:

1. when p or q is large relative to n, sample covariance matrices are singular and regularized or sparse CCA is needed;
2. matrix inverse square roots should be computed by stable factorizations, not by explicit inverse construction; and
3. successive canonical variates are uncorrelated within each block under the fitted covariance structure, but are not generally independent unless stronger distributional assumptions hold.

12 Low-rank factorization and latent-variable models

12.1 Unconstrained and nonnegative factorization

A low-rank factorization represents

$$\mathbf{X} \approx \mathbf{W}\mathbf{H}, \qquad \mathbf{W} \in \mathbb{R}^{m\times K}, \quad \mathbf{H} \in \mathbb{R}^{K\times n},$$

with $K \ll \min(m, n)$. Truncated SVD gives the best unconstrained rank-K approximation under Frobenius loss.

For nonnegative data, nonnegative matrix factorization solves, for example,

$$\min_{\mathbf{W}\geq 0,\, \mathbf{H}\geq 0} \frac{1}{2} \|\mathbf{X} - \mathbf{W}\mathbf{H}\|_F^2 .$$

Other losses, such as generalized Kullback–Leibler divergence, may be better matched to count or intensity data.

NMF can yield interpretable parts-based representations in gene expression, cell-type mixtures, or image intensities, but interpretation is not automatic:

- the problem is jointly nonconvex, so different initializations may reach different local solutions;
- the factorization has scale and permutation nonidentifiability; and
- rank selection and stability should be assessed, preferably with resampling or held-out reconstruction.

12.2 Factor analysis

A Gaussian factor model for centered p-variate observations is

$$\boldsymbol{y}_i = \boldsymbol{\Lambda}\boldsymbol{\eta}_i + \boldsymbol{\epsilon}_i, \qquad \boldsymbol{\eta}_i \sim N_K(\mathbf{0}, \mathbf{I}_K), \qquad \boldsymbol{\epsilon}_i \sim N_p(\mathbf{0}, \boldsymbol{\Psi}),$$

where $\boldsymbol{\Psi}$ is usually diagonal. Marginally,

$$\mathrm{Cov}(\boldsymbol{y}_i) = \boldsymbol{\Lambda}\boldsymbol{\Lambda}^{\mathsf{T}} + \boldsymbol{\Psi}.$$

Unlike PCA, factor analysis is a probability model that separates shared variation from variable-specific residual variation. The loading matrix is rotationally nonidentifiable without additional constraints or conventions.

Nonlinear latent-variable models replace $\boldsymbol{\Lambda}\boldsymbol{\eta}_i$ with $f(\boldsymbol{\eta}_i)$, using Gaussian processes, splines, or neural networks. Increased flexibility also increases the need for regularization, out-of-sample validation, and identifiability checks.

13 Clustering and distance-based dimension reduction

13.1 k-means

For observations $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n$, k-means partitions the data into clusters $C_1, \ldots, C_K$ by minimizing

$$\sum_{k=1}^{K} \sum_{i\in C_k} \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|_2^2 , \qquad \boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i\in C_k} \boldsymbol{x}_i.$$

The equivalent pairwise identity is

$$\sum_{i \in C_k} \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|_2^2 = \frac{1}{2|C_k|} \sum_{i,j \in C_k} \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2^2 .$$

```
X <- as.matrix(iris[, 1:4])

set.seed(2026)
fit_raw <- kmeans(X, centers = 3, nstart = 50)

set.seed(2026)
fit_scaled <- kmeans(scale(X), centers = 3, nstart = 50)

# Labels are used only for external evaluation
ari_raw <- aricode::ARI(fit_raw$cluster, iris$Species)
ari_scaled <- aricode::ARI(fit_scaled$cluster, iris$Species)
c(raw = ari_raw, scaled = ari_scaled)
```

Scaling changes the geometry and may change the clusters substantially. Because the objective is nonconvex, use multiple starts. In applications without known labels, assess cluster stability, sensitivity to preprocessing, and whether the data support discrete groups at all.

13.2 Metric MDS and classical MDS

Given dissimilarities d_{ij}, metric multidimensional scaling seeks low-dimensional points $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_n$ that minimize a stress criterion such as

$$\sum_{i<j} w_{ij} \left(\|\boldsymbol{z}_i - \boldsymbol{z}_j\|_2 - d_{ij} \right)^2 .$$

Sammon mapping uses weights proportional to $1/d_{ij}$, emphasizing the preservation of smaller distances.

Classical MDS, also called principal coordinates analysis (PCoA), has a direct eigendecomposition solution. Let $\mathbf{D}^{(2)}$ contain squared distances and

$$\mathbf{H} = \mathbf{I}_n - \frac{1}{n}\mathbf{1}\mathbf{1}^\mathsf{T}, \qquad \mathbf{B} = -\frac{1}{2}\mathbf{H}\mathbf{D}^{(2)}\mathbf{H}.$$

If

$$\mathbf{B} = \mathbf{U}\,\mathrm{diag}(\lambda_1, \ldots, \lambda_n)\mathbf{U}^\mathsf{T},$$

then the first K principal coordinates are

$$\mathbf{Z}_K = \mathbf{U}_K\,\mathrm{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_K}),$$

using positive eigenvalues.

```
X <- scale(as.matrix(iris[, 1:4]))
D <- as.matrix(dist(X))
n <- nrow(D)
H <- diag(n) - matrix(1 / n, n, n)
B <- -0.5 * H %*% (D^2) %*% H

es <- eigen(B, symmetric = TRUE)
```

```r
keep <- which(es$values > 1e-10)[1:2]
Z <- sweep(es$vectors[, keep, drop = FALSE],
           2, sqrt(es$values[keep]), "*")

plot(Z[, 1], Z[, 2],
     col = as.numeric(iris$Species), pch = 19,
     xlab = "PCoA 1", ylab = "PCoA 2")
```

For Euclidean distances, classical MDS and PCA are closely related. For non-Euclidean dissimilarities, such as some ecological or microbiome dissimilarities, $\mathbf{B}$ may have negative eigenvalues. Their magnitude is a diagnostic of how poorly the distances embed in Euclidean space.

13.3 Spectral clustering

Spectral clustering constructs a similarity graph, forms a graph Laplacian, embeds observations using selected Laplacian eigenvectors, and clusters the embedded points. Its performance depends strongly on the similarity function, neighborhood or bandwidth choice, Laplacian normalization, and the number of clusters. It should be taught as a pipeline whose sensitivity must be studied, not as a single automatic command.

14 Software map

Task	R	Python
Dense linear algebra	base R, `Matrix`	NumPy, SciPy
Sparse matrices	`Matrix`	`scipy.sparse`
PCA and decompositions	`prcomp`, base R	scikit-learn, NumPy
Clustering	`stats`, specialized packages	scikit-learn
Profiling	`Rprof`, `profvis`	`cProfile`, line profiler
Compiled extensions	Rcpp, RcppArmadillo	Cython, Numba, pybind11

15 Practice

Exercise 1 (Vectorization and memory)**.** *Simulate a* 5000×500 *matrix. Compare three ways to standardize its columns: an explicit loop,* `apply`*, and* `scale`*. Verify numerical agreement, benchmark runtime, and discuss temporary memory allocation.*

Exercise 2 (Stable least squares)**.** *Construct a design matrix containing two nearly collinear predictors. Compare the normal-equation, QR, and SVD solutions as collinearity increases. Report the condition number, coefficient error, and fitted-value error. Explain why coefficients can be unstable while fitted values remain comparatively stable.*

Exercise 3 (Cholesky diagnostics)**.** *Generate a positive definite covariance matrix, then gradually reduce its smallest eigenvalue toward zero. Record when* `chol` *fails and relate the failure to numerical rank and the condition number. Do not add a diagonal constant until the source of failure has been identified.*

Exercise 4 (PCA of high-dimensional biomedical data)**.** *Use a centered* $n \times p$ *matrix with* $p > n$*. Compute the first five principal components using both the covariance eigendecomposition and the direct SVD of the data matrix. Compare runtime, memory, scores up to sign, and explained variance.*

Exercise 5 (Preprocessing leakage)**.** *Create a prediction split, then compare two pipelines: PCA fit to all subjects before splitting, and PCA fit only in the training data. Quantify the difference in validation performance and explain why the first pipeline is optimistically biased.*

Exercise 6 (CCA in a high-dimensional regime)**.** *Simulate two data blocks with a small number of shared latent factors and with $p \geq n$ or $q \geq n$. Show why ordinary CCA is ill posed. Compare two regularization levels and evaluate the stability of the leading canonical correlation under bootstrap resampling.*

Exercise 7 (NMF stability)**.** *Fit an NMF model to a nonnegative data matrix using several random initializations and two ranks. Compare reconstruction error and factor similarity across runs. Discuss which features appear stable enough for scientific interpretation.*

Exercise 8 (Clustering and scaling)**.** *Run k-means on raw and standardized versions of a multivariate dataset. Use multiple starts, compare within-cluster sums of squares, and assess stability under bootstrap resampling. If external labels are available, use them only after fitting.*

Exercise 9 (PCoA and non-Euclidean distance)**.** *Compute PCoA from a Euclidean distance matrix and verify agreement with PCA scores up to rotation and sign. Repeat with a non-Euclidean dissimilarity and inspect the negative eigenvalues.*

References

[1] J. E. Gentle. *Computational Statistics*. Springer, 2009.

[2] G. H. Golub and C. F. Van Loan. *Matrix Computations*, 4th edition. Johns Hopkins University Press, 2013.

[3] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd edition. SIAM, 2002.

[4] K. Lange. *Numerical Analysis for Statisticians*, 2nd edition. Springer, 2010.

[5] J. F. Monahan. *Numerical Methods of Statistics*, 2nd edition. Cambridge University Press, 2011.